

R Programming for Data Analysis

DAY 1 — Introduction to R & RStudio

1. Introduction to R

What is R?

R is an open-source programming language and software environment used for statistical computing, data analysis, and data visualization.

R was originally developed for statistical analysis, but today it is widely used in the **Data Analytics, Data Science, Machine Learning, and Research** fields.

A Data Analyst can use R to:

- Import data from CSV, Excel, databases, etc.
- Clean and prepare data
- Perform calculations
- Analyze large datasets
- Find patterns and trends
- Create charts and visualizations
- Perform statistical analysis
- Build predictive models
- Generate analytical reports

Simple Example

Suppose a company has sales for five months:

```
sales <- c(15000, 18000, 22000, 19000, 25000)
```

R can quickly calculate:

```
sum(sales)
mean(sales)
max(sales)
min(sales)
```

So instead of manually calculating the values, R performs the analysis automatically.

2. Why Learn R for Data Analysis?

Data Analysts commonly work with large amounts of data.

For example, a company may have:

```
1,00,000 Customers  
5,00,000 Orders  
10,00,000 Transactions
```

Doing calculations manually is difficult and time-consuming.

R allows us to **automate the analysis**.

Example

Instead of manually calculating average salary:

```
₹25,000  
₹32,000  
₹28,000  
₹45,000  
₹38,000  
... 
```

We can write:

```
mean(salary)
```

R calculates the result instantly.

3. Applications of R

R is used in many areas.

1. Data Analysis

Used to analyze business data and find useful information.

Example:

```
Sales Analysis  
Customer Analysis  
Employee Analysis  
Financial Analysis
```

2. Data Visualization

R can create:

- Bar charts
- Line charts
- Pie charts
- Histograms
- Scatter plots
- Box plots

3. Statistics

R is particularly strong in statistical analysis.

Examples:

- Mean
- Median
- Standard deviation
- Correlation
- Regression
- Hypothesis testing

4. Machine Learning

R can also be used for:

- Linear Regression
- Logistic Regression
- Decision Trees
- Random Forest
- Clustering
- Classification

5. Reporting

R can generate automated reports and analytical documents.

4. R vs Excel vs Power BI

A Data Analyst may use several tools together.

Feature	Excel	Power BI	R
Data Entry	★★★★	★★	★
Data Cleaning	★★★★★	★★★★★	★★★★★
Data Analysis	★★★★★	★★★★★	★★★★★
Visualization	★★★★★	★★★★★	★★★★★
Statistics	★★★★★	★★★★★	★★★★★
Automation	★★★★★	★★★★★	★★★★★
Programming	Limited	Limited	★★★★★

Important

Learning R does **not** mean that Excel or Power BI becomes unnecessary.

A professional Data Analyst may use:

Excel → SQL → Power BI → R/Python

depending on the requirement.

5. What is RStudio?

RStudio is an Integrated Development Environment (IDE) for R.

It provides a convenient interface for writing, executing, debugging, and managing R programs.

Think of it this way:

R = Programming Language

RStudio = Software used to work with R

6. RStudio Interface

RStudio generally contains four important areas.

1. Source / Script

Used to write and save R programs.

Example:

```
sales <- c(10000, 20000, 30000)

sum(sales)
```

2. Console

The Console is where R commands are executed.

Example:

```
10 + 20
```

Output:

```
30
```

3. Environment

The Environment displays objects/variables that have been created.

For example:

```
sales <- c(10000, 20000, 30000)
```

RStudio will show `sales` in the Environment.

4. Files / Plots / Packages / Help

This area can be used for:

- Managing files
- Viewing charts
- Installing/managing packages

- Reading documentation
 - Getting help about functions
-

7. First R Program

Let's write our first program.

```
print("Hello World")
```

Output:

```
[1] "Hello World"
```

Another way:

```
"Hello World"
```

8. Comments in R

A **comment** is a note written inside the program.

Comments are ignored by R.

Use:

```
#
```

Example:

```
# This is my first R program

print("Hello World")
```

Comments help us understand our code later.

Example

```
# Store monthly salary
salary <- 35000

# Calculate annual salary
annual_salary <- salary * 12
```

9. Variables in R

A variable is a named storage location used to hold a value.

Example:

```
name <- "Amit"  
age <- 25  
salary <- 40000
```

Here:

- name stores "Amit"
- age stores 25
- salary stores 40000

Display them:

```
name  
age  
salary
```

10. Assignment Operator

The most commonly used assignment operator in R is:

```
<-
```

Example:

```
salary <- 40000
```

It means:

Store the value 40000 inside the variable salary.

Another example:

```
city <- "Mumbai"
```

11. Rules for Naming Variables

Correct

```
student_name <- "Rahul"  
student_age <- 21  
monthly_salary <- 35000
```

Incorrect

```
student name <- "Rahul"
```

Spaces should not normally be used in variable names.

Instead:

```
student_name <- "Rahul"
```

R is case-sensitive

These are different:

```
salary  
Salary  
SALARY
```

For example:

```
salary <- 30000  
Salary <- 50000
```

These are two different variables.

12. Data Types in R

Data type tells us **what kind of information a variable contains**.

Important basic data types are:

1. Character
2. Numeric
3. Integer
4. Logical

12.1 Character

Character data represents text.

```
name <- "Umesh"  
city <- "Mumbai"  
course <- "Data Analysis"
```

Check the type:

```
class(name)
```

Output:

```
"character"
```

12.2 Numeric

Numeric data represents numbers.

```
salary <- 45000  
percentage <- 85.5  
price <- 999.99
```

Check:

```
class(salary)
```

12.3 Integer

An integer is a whole number.

```
age <- 25L
```

The `L` indicates an integer value.

12.4 Logical

Logical data has two values:

```
TRUE  
FALSE
```

Example:

```
is_employee <- TRUE  
is_manager <- FALSE
```

Logical values are extremely important when filtering data.

13. Checking Data Type

Use:

```
class()
```

Example:

```
name <- "Amit"
age <- 25
salary <- 45000
is_employee <- TRUE

class(name)
class(age)
class(salary)
class(is_employee)
```

14. Arithmetic Operators

R can perform mathematical calculations.

```
10 + 5
10 - 5
10 * 5
10 / 5
10 ^ 2
10 %% 3
```

Operators

Operator	Meaning
+	Addition
-	Subtraction

*	Multiplication
/	Division
^	Power
%%	Remainder

15. Practical Example — Salary Calculation

Suppose an employee earns ₹40,000 per month.

```
monthly_salary <- 40000
```

Calculate annual salary:

```
annual_salary <- monthly_salary * 12
```

Display:

```
annual_salary
```

Output:

```
480000
```

Interpretation

Monthly salary = ₹40,000

Annual salary = ₹4,80,000

16. Comparison Operators

Comparison operators compare two values.

They return:

```
TRUE
```

or

```
FALSE
```

Example:

```
age <- 25
```

```
age > 18
```

Output:

```
TRUE
```

Operators

Operator	Meaning
>	Greater than
<	Less than
==	Equal to
!=	Not equal to
>=	Greater than or equal
<=	Less than or equal

Important

For comparison, use:

```
==
```

Example:

```
age == 25
```

Don't confuse it with assignment:

```
age <- 25
```

17. Functions in R

A function is a reusable block of code that performs a particular task.

Example:

```
sum()
```

calculates a total.

```
mean()
```

calculates an average.

18. Important Functions for Data Analysis

```
sum()
```

Calculates total.

```
sales <- c(10000, 15000, 20000)

sum(sales)
```

Output:

```
45000
```

```
mean()
```

Calculates average.

```
mean(sales)
```

Output:

```
15000
```

`min()`

Finds the smallest value.

```
min(sales)
```

`max()`

Finds the largest value.

```
max(sales)
```

`length()`

Counts the number of values.

```
length(sales)
```

19. Creating a Vector

A **vector** is a collection of values of the same basic type.

Use the `c()` function.

`c` means **combine**.

Example:

```
sales <- c(10000, 15000, 12000, 18000, 20000)
```

Now we have five sales values stored in `sales`.

We can analyze them:

```
sum(sales)
mean(sales)
min(sales)
max(sales)
length(sales)
```

20. Complete Data Analyst Example

Suppose an e-commerce company has the following monthly sales:

```
sales <- c(25000, 32000, 28000, 35000, 42000, 39000)
```

Total Sales

```
total_sales <- sum(sales)
total_sales
```

Average Sales

```
average_sales <- mean(sales)
average_sales
```

Highest Sales

```
highest_sales <- max(sales)
highest_sales
```

Lowest Sales

```
lowest_sales <- min(sales)
lowest_sales
```

Number of Records

```
number_of_months <- length(sales)
number_of_months
```

Check Average Sales

```
average_sales > 30000
```

This returns `TRUE` or `FALSE` .

DAY 1 CLASSROOM PRACTICE

Practice 1 — Student Information

Create variables for:

```
Student Name = Rahul
Age = 22
Course = Data Analysis
Fees = 25000
```

Display all values and check their data types.

Practice 2 — Employee Salary

Create:

```
employee <- "Amit"  
monthly_salary <- 35000
```

Calculate:

1. Annual salary
 2. Salary for 2 years
 3. Check whether monthly salary is greater than ₹30,000
-

Practice 3 — Sales Analysis

Create:

```
sales <- c(15000, 18000, 22000, 19000, 25000)
```

Find:

1. Total sales
 2. Average sales
 3. Maximum sales
 4. Minimum sales
 5. Number of sales records
-

Practice 4 — Marks Analysis

```
marks <- c(65, 78, 82, 55, 90, 72, 88)
```

Find:

1. Total marks
 2. Average marks
 3. Highest marks
 4. Lowest marks
 5. Number of students
-



DAY 1 HOME ASSIGNMENT

A company has the following monthly revenue:

```
revenue <- c(45000, 52000, 48000, 61000, 58000, 70000, 65000)
```

Write an R program to calculate:

1. Total Revenue
2. Average Revenue
3. Highest Revenue
4. Lowest Revenue
5. Number of Months
6. Check whether average revenue is greater than ₹55,000
7. Check whether highest revenue is greater than ₹65,000